MODUL 8

Binary Search Tree

Modul Praktikum Struktur Data • C++17

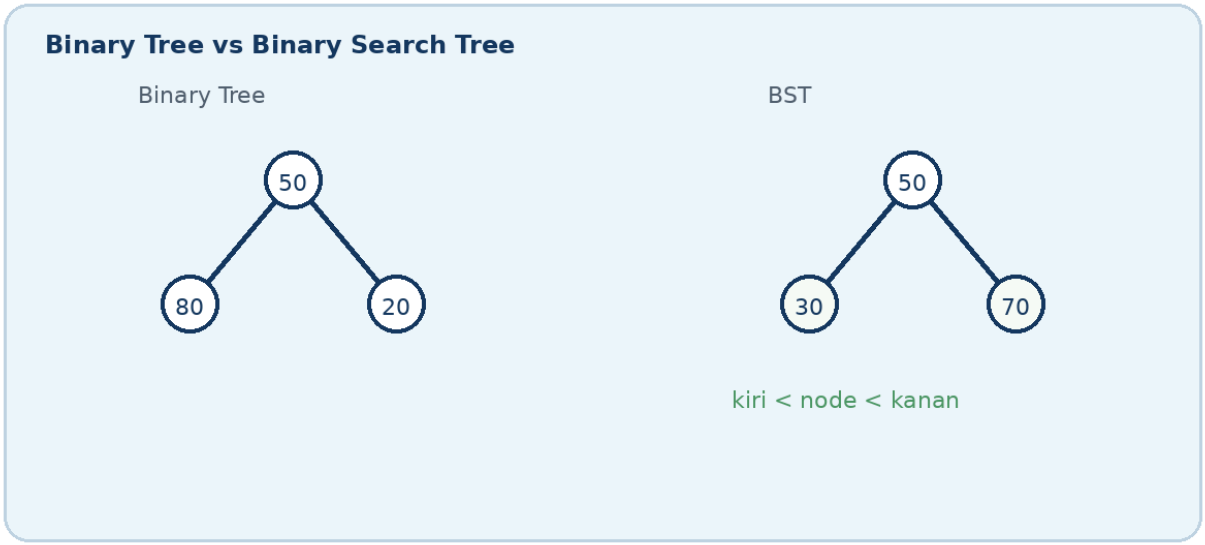
Program Studi Informatika • Praktikum Struktur Data • C++17

1. Tujuan Praktikum

1. menjelaskan perbedaan binary tree dan Binary Search Tree (BST);
2. menjelaskan invariant pengurutan pada BST;
3. melakukan pencarian berdasarkan perbandingan nilai;
4. menambahkan node sambil mempertahankan invariant BST;
5. menolak nilai duplikat sesuai kebijakan modul;
6. menggunakan inorder traversal sebagai pemeriksaan praktis terhadap urutan nilai;
7. menemukan nilai minimum pada suatu subtree;
8. menghapus node dengan 0 child, 1 child, dan 2 child;
9. menggunakan inorder successor pada penghapusan node dengan 2 child; dan
10. menguji bahwa BST tetap valid setelah insertion dan deletion.

2. Dari Binary Tree ke Binary Search Tree

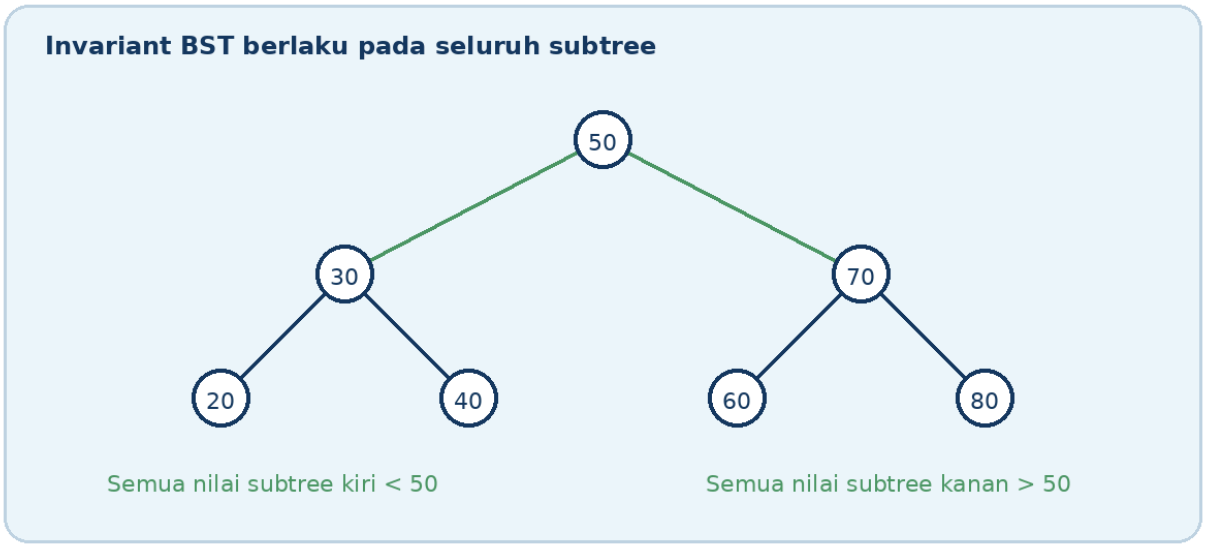
Binary tree hanya membatasi setiap node memiliki paling banyak dua child. Binary Search Tree menambahkan aturan pengurutan sehingga pencarian dan perubahan struktur dapat diarahkan berdasarkan nilai.



Gambar 1. Binary tree dan Binary Search Tree.

2.1 Invariant BST

Untuk setiap node, semua nilai pada subtree kiri harus lebih kecil daripada nilai node, sedangkan semua nilai pada subtree kanan harus lebih besar. Aturan ini berlaku terhadap seluruh subtree, bukan hanya child langsung.



Gambar 2. Invariant BST berlaku terhadap seluruh subtree.

Pada modul ini, nilai duplikat ditolak agar aturan insertion, search, dan deletion tetap konsisten.

2.2 Struktur node

```
struct Node {
    int data;
    Node* left;
    Node* right;
};
```

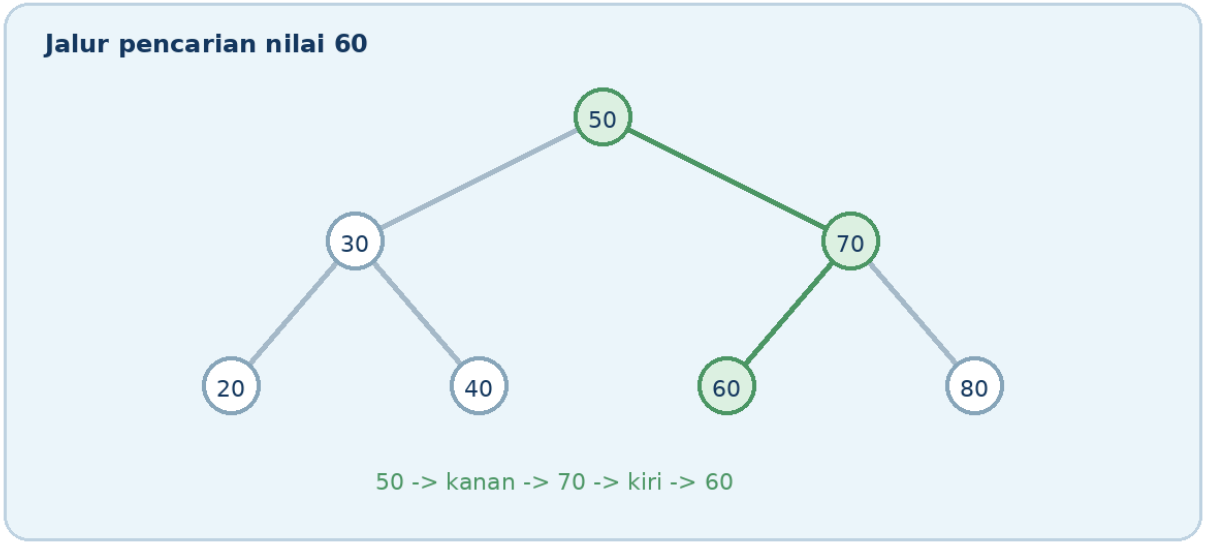
```
Node* root = nullptr;
```

Struktur node dan pointer root sama seperti pada Modul 7. Pada Modul 8, perhatian utama beralih dari traversal menuju pemanfaatan dan pemeliharaan invariant BST.

3. Operasi Binary Search Tree

3.1 Search

Pencarian membandingkan nilai yang dicari dengan data pada node saat ini. Jika lebih kecil, pencarian dilanjutkan ke subtree kiri. Jika lebih besar, pencarian dilanjutkan ke subtree kanan. Jika sama, node ditemukan.



Gambar 3. Jalur pencarian nilai 60 pada BST.

```
const Node* cari(const Node* root, int nilai) {
    if (root == nullptr)
        return nullptr;

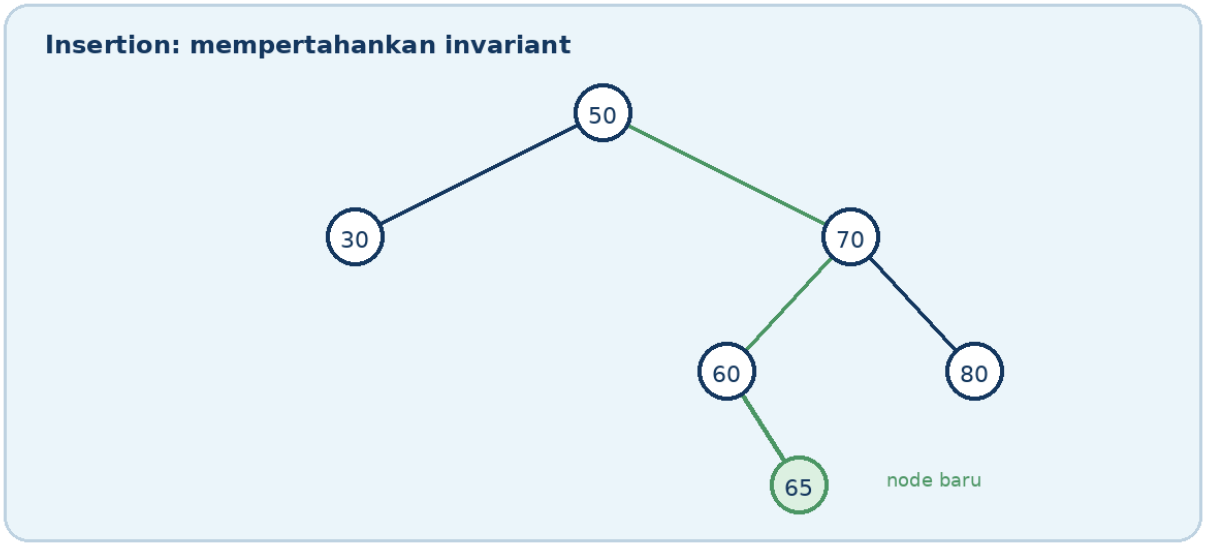
    if (nilai == root->data)
        return root;

    if (nilai < root->data)
        return cari(root->left, nilai);

    return cari(root->right, nilai);
}
```

3.2 Insert

Insertion menggunakan keputusan yang sama dengan search. Node baru ditempatkan pada posisi nullptr yang sesuai sehingga invariant BST tetap terpenuhi.



Gambar 4. Insertion nilai 65 mengikuti jalur pencarian hingga posisi kosong.

```
bool insert(Node*& root, int nilai) {
    if (root == nullptr) {
        root = new Node{nilai, nullptr, nullptr};
        return true;
    }

    if (nilai < root->data)
        return insert(root->left, nilai);

    if (nilai > root->data)
        return insert(root->right, nilai);

    return false; // duplikat ditolak
}
```

3.3 Inorder sebagai pemeriksaan

Fungsi inorder dari Modul 7 digunakan kembali untuk memeriksa keadaan BST. Pada BST valid tanpa duplikasi, inorder menghasilkan nilai dalam urutan naik. Pemeriksaan ini membantu, tetapi praktikan tetap harus memahami invariant pada seluruh subtree.

```
void inorder(const Node* root) {
    if (root == nullptr)
        return;

    inorder(root->left);
    std::cout << root->data << ' ';
    inorder(root->right);
}
```

3.4 Nilai minimum

Nilai minimum sebuah BST atau subtree ditemukan dengan bergerak terus ke child kiri sampai tidak ada child kiri lagi. Fungsi ini juga digunakan pada deletion dua child.

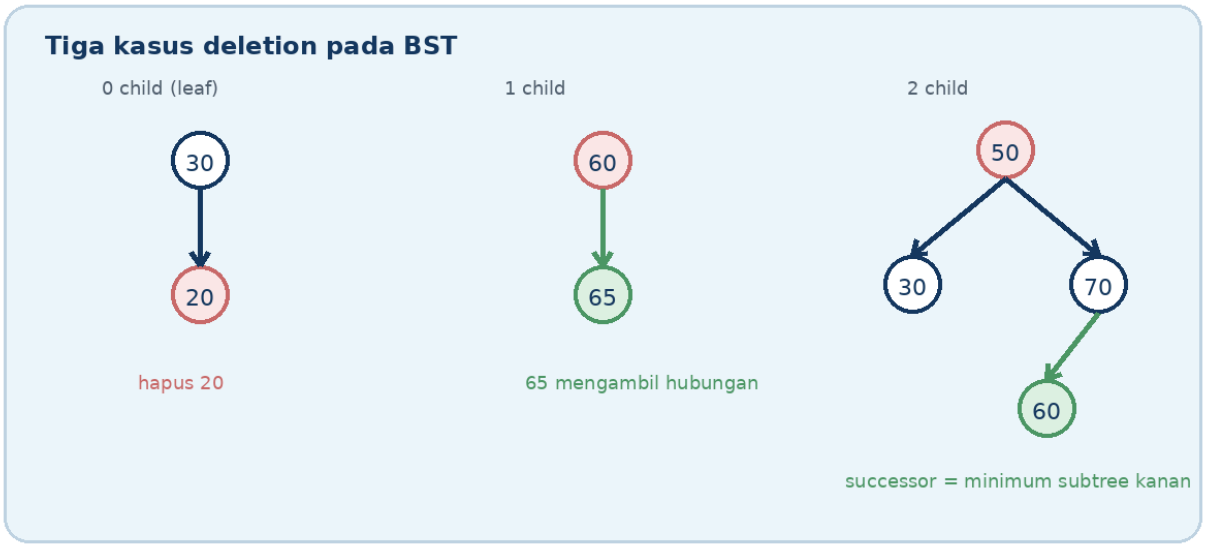
```
const Node* minimum(const Node* root) {
    if (root == nullptr)
        return nullptr;

    while (root->left != nullptr)
        root = root->left;

    return root;
}
```

4. Deletion pada Binary Search Tree

Deletion harus menghapus target sekaligus mengembalikan tree ke keadaan yang tetap memenuhi invariant BST. Terdapat tiga kasus utama.



Gambar 5. Tiga kasus utama deletion pada BST.

4.1 Menghapus node dengan 0 child

Node leaf tidak mempunyai child. Setelah target ditemukan, node dapat dibebaskan dan pointer subtree dibuat nullptr.

```
Node* target = root;
root = nullptr;
delete target;
```

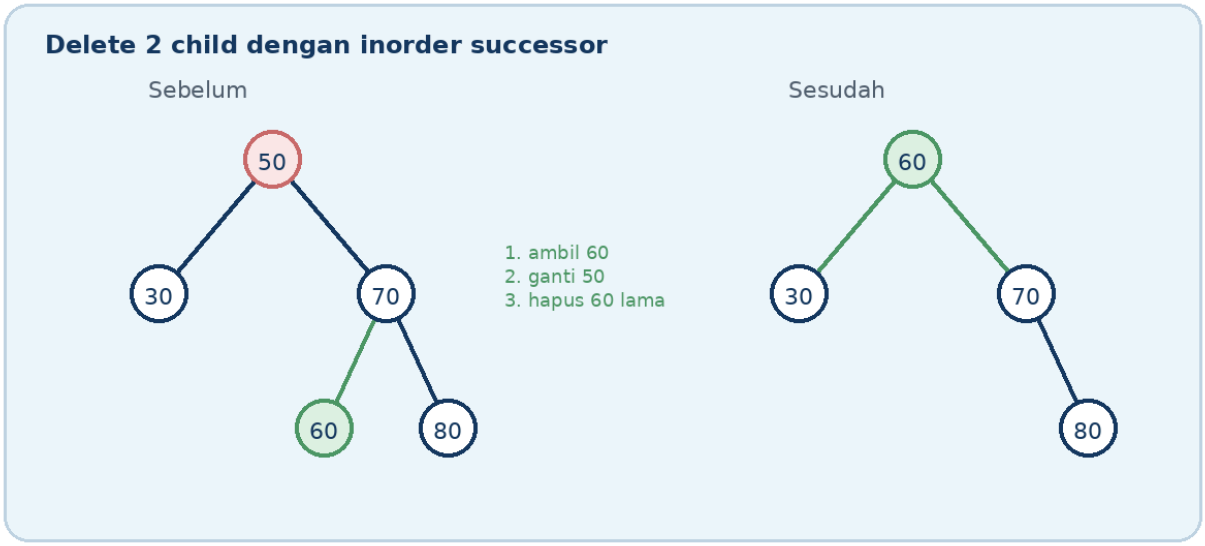
4.2 Menghapus node dengan 1 child

Jika target hanya mempunyai satu child, pointer subtree target diganti dengan child tersebut sebelum target dibebaskan.

```
Node* target = root;
root = (root->left != nullptr) ? root->left : root->right;
delete target;
```

4.3 Menghapus node dengan 2 child

Jika target mempunyai dua child, modul ini menggunakan inorder successor sebagai pengganti. Inorder successor adalah node minimum pada subtree kanan.



Gambar 6. Delete dua child menggunakan inorder successor.

1. cari nilai minimum pada subtree kanan;
2. salin nilai successor ke node target;
3. hapus node successor lama dari subtree kanan; dan
4. periksa kembali invariant BST.

```
const Node* pengganti = minimum(root->right);
root->data = pengganti->data;
hapus(root->right, pengganti->data);
```

4.4 Fungsi hapus

Fungsi berikut menunjukkan struktur keputusan deletion. Lengkapi bagian yang belum selesai pada kegiatan praktikum.

```
bool hapus(Node*& root, int nilai) {
    if (root == nullptr)
        return false;

    if (nilai < root->data) {
        return hapus(root->left, nilai);
    } else if (nilai > root->data) {
        return hapus(root->right, nilai);
    } else {
        // target ditemukan

        if (root->left == nullptr && root->right == nullptr) {
            // kasus 0 child
        }
```

```
        if (root->left == nullptr) {
            // hanya child kanan
        }

        if (root->right == nullptr) {
            // hanya child kiri
        }

        // kasus 2 child
        // gunakan minimum pada subtree kanan
    }

    return true;
}
```

5. Kegiatan Praktikum

Gunakan satu berkas bernama latihan_bst.cpp. Kompilasi program setelah setiap operasi selesai ditambahkan.

5.1 Membentuk BST

1. buat root dengan nilai awal nullptr;
2. masukkan nilai 50, 30, 70, 20, 40, 60, dan 80 menggunakan insert();
3. tampilkan inorder; dan
4. pastikan hasilnya 20 30 40 50 60 70 80.

5.2 Search dan insert

1. cari nilai 60 dan catat jalur pencariannya;
2. cari nilai 25 dan periksa hasil tidak ditemukan;
3. insert nilai 65;
4. coba insert nilai 70 kembali dan pastikan ditolak; dan
5. jalankan inorder setelah insertion.

5.3 Minimum

1. cari minimum seluruh tree;
2. cari minimum pada subtree kanan root; dan
3. jelaskan mengapa fungsi selalu bergerak ke kiri.

5.4 Deletion

1. hapus leaf 20;
2. siapkan state sehingga node 60 mempunyai satu child 65, kemudian hapus 60;
3. hapus node dengan dua child, misalnya 70 atau root 50;
4. jalankan inorder setelah setiap deletion; dan

5. periksa bahwa nilai tetap terurut serta target benar-benar hilang.

6. Tugas Praktikum

Buat program Pengelolaan Binary Search Tree dan simpan dalam modul8_bst.cpp. Program menggunakan data integer dan menyediakan menu berikut:

1. Tambah nilai
2. Cari nilai
3. Hapus nilai
4. Tampilkan inorder
5. Tampilkan minimum
6. Keluar

Nilai duplikat harus ditolak. Sebelum program berakhir, gunakan fungsi clear() dari Modul 7 untuk membebaskan seluruh node.

6.1 Kerangka program

```
#include <iostream>

struct Node {
    int data;
    Node* left;
    Node* right;
};

bool insert(Node*& root, int nilai);
const Node* cari(const Node* root, int nilai);
const Node* minimum(const Node* root);
bool hapus(Node*& root, int nilai);
void inorder(const Node* root);
void clear(Node*& root);

int main() {
    Node* root = nullptr;
    int pilihan;

    do {
        std::cout << "\n1. Tambah\n";
        std::cout << "2. Cari\n";
        std::cout << "3. Hapus\n";
        std::cout << "4. Tampilkan Inorder\n";
        std::cout << "5. Minimum\n";
        std::cout << "6. Keluar\n";
        std::cout << "Pilihan: ";
        std::cin >> pilihan;

        // panggil fungsi sesuai pilihan

    } while (pilihan != 6);
```

```
    clear(root);
    return 0;
}
```

7. Pengujian

No.	Skenario	Hasil yang diharapkan
1	BST baru	root == nullptr
2	Insert pertama	Nilai menjadi root
3	Insert beberapa nilai	Posisi sesuai invariant
4	Insert duplikat	Ditolak; tree tidak berubah
5	Search tersedia	Node ditemukan
6	Search tidak tersedia	nullptr / tidak ditemukan
7	Minimum	Node paling kiri
8	Delete leaf	Target hilang; tree valid
9	Delete 1 child	Child mengambil hubungan target
10	Delete 2 child	Target diganti inorder successor
11	Delete nilai tidak tersedia	Tree tidak berubah
12	Inorder setelah perubahan	Tetap terurut naik

Setelah setiap insertion dan deletion, periksa bentuk tree pada bagian yang berubah dan jalankan inorder sebagai pemeriksaan praktis.

7.1 Diagnosis kode

Setiap potongan berikut mengandung kesalahan. Tentukan kesalahan, akibat, dan perbaikannya.

A. Arah search terbalik

```
if (nilai < root->data)
    return cari(root->right, nilai);
```

B. Insert melanggar ordering

```
if (nilai < root->data)
    return insert(root->right, nilai);
```

C. Duplikat selalu diarahkan ke kanan

```
if (nilai < root->data)
    insert(root->left, nilai);
else
    insert(root->right, nilai);
```

D. Minimum bergerak ke kanan

```
while (root->right != nullptr)
    root = root->right;
```

E. Delete leaf menyisakan dangling pointer

```
delete root;
return true;
```

F. Delete satu child membuang child

```
Node* target = root;
root = nullptr;
delete target;
```

G. Delete dua child tidak menghapus successor lama

```
const Node* pengganti = minimum(root->right);
root->data = pengganti->data;
return true;
```

8. Pengayaan

• membuat fungsi maximum();

• menggunakan fungsi height() dari Modul 7 untuk mengamati bentuk BST;

• membandingkan bentuk BST yang dihasilkan dari urutan insertion berbeda.

9. Jembatan ke Modul 9

Binary tree membatasi hubungan dalam struktur hierarkis. Pada modul berikutnya, Graph memperluas gagasan hubungan: sebuah vertex dapat terhubung dengan banyak vertex lain tanpa harus membentuk struktur parent-child.